Clint Scott

CSD 380 – DevOps

Module 6.2 Assignment – Case Study: Strangler Pattern at Blackboard Learn (2011)

10/25/25

Strangler Pattern at Blackboard Learn

Summary of Main Points

In 2011, Blackboard Learn encountered significant issues with its primary Learning Management System (LMS). The platform had evolved into a massive, complex monolith with code dating back to 1997, still relying on outdated technologies like Perl. This technical debt made the system incredibly fragile. Even minor code tweaks required the team to rebuild the entire system, which made deployments slow and hazardous. Because the teams were so tightly coupled, a change in one part of the app would frequently break something totally unrelated elsewhere.

A review of the repository statistics from 2005 to 2010 revealed a concerning trend: the code size was increasing rapidly, but the number of actual commits was decreasing. This proved that productivity was stalling; it was becoming increasingly complex to build new features or fix bugs. To make matters worse, developers were forced to wait 24 to 36 hours for integration and testing feedback, which hindered their momentum and increased frustration.

To address this, Blackboard adopted the Strangler Pattern. This approach enabled them to modernize the system piece by piece, rather than attempting a risky total rewrite. They reorganized the system into "Building Blocks", independent modules with strict APIs. This allows developers to work independently without stepping on each other's toes or wrestling with

the monolith. They set up a "facade" layer to sit in front of the legacy system, routing specific requests to the new modules while letting standard traffic flow to the old code.

Eventually, they moved more functionality over to the new architecture. This reduced the risk of crashes, expedited their work, and enabled them to deliver value to customers immediately. The developers preferred working on the Building Blocks because failures were contained; a bad deploy wouldn't take down the whole LMS, making their day-to-day work much safer and more productive.

Lessons Learned

This case study offers a few critical takeaways for any organization trying to manage a massive legacy codebase.

First, the Strangler Pattern proves that chipping away at modernization is far safer than trying a total rewrite. "Big Bang" rewrites are notorious for delaying value and carrying huge risks, whereas an incremental approach allows for continuous progress without risking the company on a single release.

Second, breaking the system into modular components stops teams from getting bottlenecked by coordination issues. This directly supports Conway’s Law: if you want your teams to work independently and move fast, your architecture needs to be decoupled enough to allow it.

Third, isolation limits the "blast radius" of mistakes. When modules are separate, teams can test, ship, or roll back a specific feature without worrying that they will crash the entire platform or negatively impact other users.

Fourth, data matters. Keeping an eye on specific metrics, such as the ratio of lines of code to commit frequency, can serve as an early warning system for declining system health and stalling developer productivity.

Finally, organizations need to stop looking at legacy systems as obstacles that need to be destroyed immediately. Instead, they should be treated as a foundation to be improved over time. This mindset keeps the business running while slowly paying down technical debt, rather than pausing everything for a rewrite that might never happen.

References

Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations* (2nd ed.). IT Revolution.